

Software Design Document
AI-Based Image-Based Motorcycle Helmet Detection



Submitted by:

Hassan Javed

Roll No.:

S23BINFT1M01011 (1M)

Submitted to:

Mr.Munawar Ahmad

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur

Summary

The AI-Based Image-Based Motorcycle Helmet Detection System is an intelligent computer vision-based solution designed to automatically detect motorcycle riders and determine whether they are wearing safety helmets. The system utilizes YOLO-based deep learning, OpenCV image processing, and real-time video analysis to identify helmet violations from images, recorded videos, and live CCTV feeds. Its primary purpose is to support traffic authorities in enforcing road safety regulations by reducing manual monitoring efforts and improving detection efficiency.

The system follows a modular and layered architecture, including input acquisition, processing, detection, violation management, and storage components. It uses an iterative development methodology to continuously improve model performance and system reliability. The project also incorporates dataset collection, annotation, preprocessing, and augmentation to enhance model accuracy.

A structured database is used to store detection records, violation logs, and camera information, enabling reporting and historical analysis. The user interface is designed to be simple and responsive, providing dashboards, live monitoring, detection results, violation logs, reports, and system settings.

Validation rules such as email verification, camera availability checks, detection threshold limits, and file format validation ensure system stability, accuracy, and user-friendly operation. Overall, the system provides an efficient, scalable, and real-time solution for automated motorcycle helmet detection and traffic violation monitoring.

Table of Contents

1. Introduction.....	3
1.1 Project Overview.....	3
1.2 Purpose of the System.....	3
1.3 Software Design Description (SDD) Overview.....	3
1.4 System Architecture Overview.....	3
1.5 Design Objectives.....	3
1.6 Scope of the System.....	4
2. Methodology.....	4
2.1 System Development Methodology.....	4
2.2 Data Collection and Preparation.....	4
2.3 Model Training Methodology.....	5
2.4 Detection and Classification Workflow.....	5
3. Application Architecture.....	6
3.1 Architectural Overview.....	6
3.1.1 Input Layer (Image/Video Acquisition).....	8
3.1.2 Processing Layer.....	9
3.1.3 Detection Layer.....	9
3.1.4 Violation Management Layer.....	10
3.1.5 Storage and Logging Layer.....	11
3.2 List of Proposed Modules & Functions.....	11
3.3 External Libraries & Frameworks.....	14
3.4 Coding Conventions.....	16
4. Data Model Schema.....	17
4.1 Database Overview.....	17
4.2 Entity Descriptions.....	18
4.3 Entity Relationship Diagram (ERD).....	21
5. User Interface Design.....	21
5.1 List of Core Pages & Control Details.....	23
5.2 Input Constraints & Validation Rules.....	29
6. References.....	31

1. Introduction

1.1 Project Overview

The AI-Based Image-Based Motorcycle Helmet Detection System is an intelligent traffic monitoring solution designed to automatically identify whether motorcycle riders are wearing safety helmets.

1.2 Purpose of the System

The primary goal of the system is to assist traffic authorities in enforcing road safety regulations through automated detection and monitoring of helmet violations. It reduces the need for manual surveillance and improves monitoring efficiency.

1.3 Software Design Description (SDD) Overview

This Software Design Description (SDD) presents the technical design and implementation approach of the proposed system. It serves as a blueprint for transforming the requirements specified in the Software Requirements Specification (SRS) into a functional software solution.

1.4 System Architecture Overview

The system follows a modular architecture consisting of image acquisition, motorcycle detection, helmet classification, violation management, result visualization, and data logging modules.

1.5 Design Objectives

The software design emphasizes:

- Scalability
- Maintainability
- Reliability
- Accuracy
- Real-Time Performance

1.6 Scope of the System

The system supports the analysis of images, recorded videos, and live CCTV feeds to detect motorcycles, identify helmet usage, and generate violation records for traffic monitoring and enforcement.

2. Methodology

The development of the AI-Based Image-Based Motorcycle Helmet Detection System follows a systematic approach that combines data collection, model training, image processing, and real-time detection techniques. The methodology is designed to ensure accurate helmet detection, efficient processing, and reliable performance in various traffic monitoring environments.

2.1 System Development Methodology

The system is developed using an iterative development approach, where individual modules are designed, implemented, tested, and refined independently before integration into the complete system. This approach allows continuous improvement of the detection model and facilitates easier debugging and maintenance throughout the development process.

2.2 Data Collection and Preparation

The performance of the detection model depends heavily on the quality of the training dataset. Images containing motorcycle riders with and without helmets are collected from publicly available datasets such as Kaggle, Roboflow, and other open-source repositories. The collected images are annotated using labeling tools to identify motorcycles, riders, helmets, and non-helmet cases.

After collection, the dataset is cleaned and preprocessed by removing duplicate or low-quality images. Data augmentation techniques such as image rotation, scaling, flipping, and brightness adjustment are applied to improve model generalization and increase detection accuracy under different environmental conditions.

2.3 Model Training Methodology

The helmet detection model is developed using the YOLO (You Only Look Once) object detection framework. The annotated dataset is divided into training, validation, and testing sets.

During training, the model learns to detect motorcycles and classify riders as either wearing a helmet or not wearing a helmet.

The training process involves multiple iterations to optimize model parameters and improve detection accuracy. Performance metrics such as precision, recall, and mean average precision (mAP) are used to evaluate the effectiveness of the trained model. The best-performing model is selected for deployment within the detection system.

2.4 Detection and Classification Workflow

The operational workflow of the AI-Based Motorcycle Helmet Detection System begins when an image, recorded video, or live CCTV camera feed is provided as input. The input data is processed using OpenCV, which extracts and prepares individual video frames for analysis. These frames are then passed to the YOLO-based object detection model, which identifies motorcycles and riders present within the scene.

After detecting a motorcycle rider, the system focuses on the rider region and performs helmet classification to determine whether the rider is wearing a safety helmet. Based on the classification result, the rider is labeled as either **“Helmet”** or **“No Helmet.”** When a helmet violation is detected, the system automatically generates an alert and records the violation details, including the detection result, timestamp, and other relevant information for future reference.

The processed output is displayed in real time with bounding boxes and classification labels over the detected objects, allowing users to easily visualize the detection results. Additionally, all detection and violation records are stored in the database or log files to support monitoring, analysis, and reporting activities. This workflow enables efficient and automated helmet detection while ensuring high accuracy, reliability, and suitability for real-time traffic surveillance and road safety enforcement applications.

3. Application Architecture

The AI-Based Image-Based Motorcycle Helmet Detection System is designed using a modular layered architecture to ensure efficient processing, scalability, and maintainability. The architecture integrates computer vision techniques, deep learning models, and data management components to perform automated helmet detection and violation monitoring.

The system begins by acquiring images or video streams from input sources such as CCTV cameras, webcams, or stored media files. The acquired frames are processed using image processing techniques to enhance detection quality and prepare the data for analysis. A trained YOLO-based deep learning model is then used to detect motorcycles and riders within each frame.

After detecting a motorcycle rider, the system performs helmet classification to determine whether the rider is wearing a helmet. Based on the classification result, the system identifies potential traffic violations and generates alerts when necessary. Detection results are displayed visually using bounding boxes and labels, while relevant information such as timestamps and violation records is stored for future analysis and reporting.

The architecture is divided into several functional layers, including the Input Layer, Processing Layer, Detection Layer, Violation Management Layer, and Storage & Logging Layer. Each layer performs a specific responsibility and communicates with other layers to ensure smooth and reliable system operation. This modular design allows future enhancements such as license plate recognition, multi-camera support, and advanced traffic analytics without significant modifications to the core system.

3.1 Architectural Overview

The AI-Based Image-Based Motorcycle Helmet Detection System follows a layered architecture in which each layer is responsible for a specific task within the detection pipeline. This design improves system organization, simplifies maintenance, and enables independent enhancement of individual modules without affecting the overall functionality of the system.

The architecture begins with the acquisition of images or live video streams from various input sources. The captured data is then forwarded to the processing layer, where image preprocessing operations are performed to improve the quality of the input data. After preprocessing, the detection layer utilizes a trained YOLO-based deep learning model to identify motorcycles and classify riders as either wearing a helmet or not wearing a helmet.

Once the classification process is completed, the violation management layer evaluates the detection results and determines whether a traffic violation has occurred. In the event of a helmet violation, the system generates alerts and prepares the violation data for storage. Finally, the storage and logging layer records all detection results, timestamps, and violation information for future monitoring, reporting, and analysis.

The separation of responsibilities among these layers ensures efficient processing, high detection accuracy, and easier integration of future features such as license plate recognition, cloud-based monitoring, and multi-camera support.

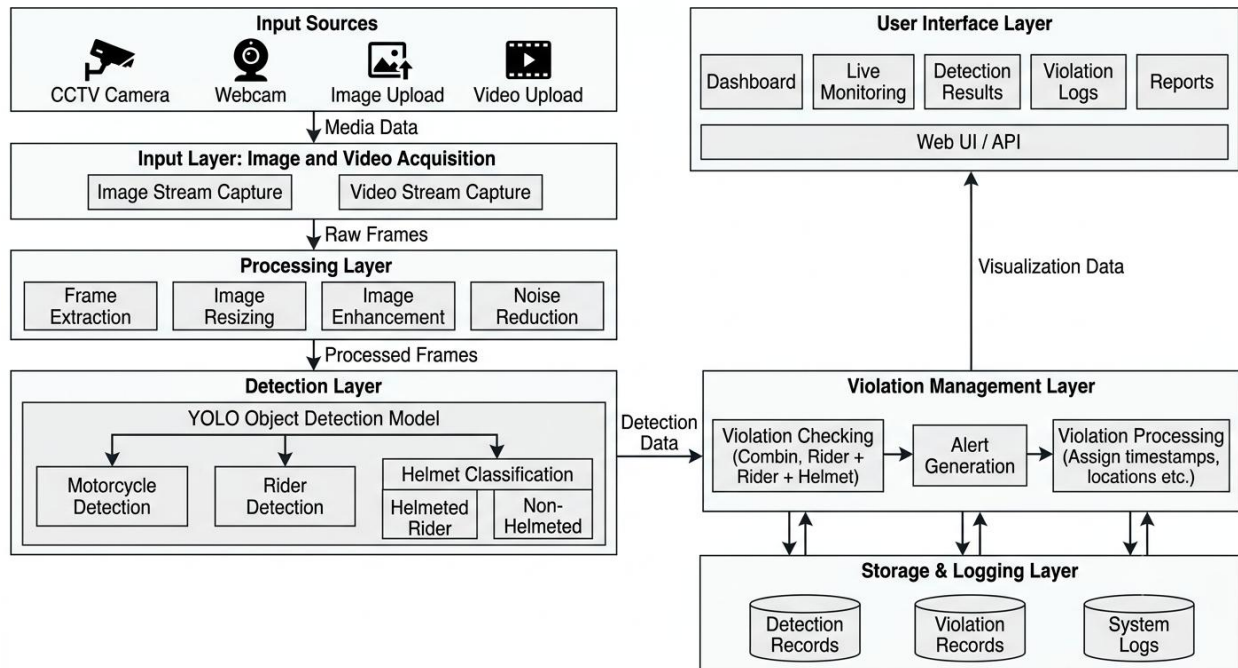


Fig 3.1: Architectural Overview

3.1.1 Input Layer (Image/Video Acquisition)

The Input Layer serves as the entry point of the Motorcycle Helmet Detection System. Its primary responsibility is to acquire visual data from various sources, including CCTV cameras, webcams, recorded video files, and static images. This layer ensures that image frames are continuously captured and made available for further processing by subsequent system components.

For real-time monitoring, the system establishes a connection with live camera feeds and continuously retrieves video frames at regular intervals. For offline analysis, users can upload images or recorded videos that are processed individually. The acquired frames are converted into a format suitable for computer vision operations and forwarded to the processing layer.

The Input Layer also performs basic validation to ensure that the incoming media source is accessible and functioning correctly. If a camera connection fails or an unsupported file format is provided, the system generates an appropriate error message and prevents invalid data from entering the detection pipeline.

By providing a consistent and reliable stream of visual data, this layer forms the foundation for accurate motorcycle detection and helmet classification throughout the system.

3.1.2 Processing Layer

The Processing Layer is responsible for preparing the acquired images and video frames for accurate object detection and classification. Since raw input data may contain noise, varying lighting conditions, motion blur, or unnecessary information, preprocessing operations are performed to improve image quality and optimize detection performance.

This layer utilizes the OpenCV library to perform various image processing tasks. Each frame captured from the input source is resized to match the input dimensions required by the YOLO model. Image normalization techniques are applied to ensure consistent pixel values, while frame extraction is performed continuously for real-time video streams. These operations help reduce computational complexity and improve processing efficiency.

In addition, the Processing Layer handles image enhancement operations such as brightness adjustment, contrast optimization, and noise reduction when necessary. These techniques improve the visibility of motorcycles and riders under challenging environmental conditions, including low-light situations, shadows, and adverse weather conditions.

After preprocessing is completed, the optimized image frame is forwarded to the Detection Layer for motorcycle detection and helmet classification. By ensuring that high-quality data is provided to the deep learning model, this layer plays a critical role in improving the overall accuracy and reliability of the system.

3.1.3 Detection Layer

The Detection Layer is the core component of the Motorcycle Helmet Detection System. It is responsible for identifying motorcycles and determining whether riders are wearing helmets by utilizing a trained YOLO-based deep learning model. This layer performs the primary analytical tasks that enable automated traffic violation detection.

When a pre processed image frame is received from the Processing Layer, the YOLO model analyzes the frame and detects objects of interest. The model identifies motorcycles and riders by generating bounding boxes around detected objects and assigning confidence scores to each prediction. Only detections that meet the predefined confidence threshold are considered valid.

After motorcycle detection, the system focuses on the rider region and performs helmet classification. The trained model examines visual features associated with helmet usage and categorizes each rider into one of two classes: “Helmet” or “No Helmet.” The classification result is displayed together with the corresponding confidence level.

The Detection Layer is designed to operate efficiently in both image-based and real-time video processing environments. Its ability to perform object detection and classification within a single processing pipeline enables rapid decision-making and supports continuous traffic monitoring applications. The output generated by this layer is then forwarded to the Violation Management Layer for further analysis and enforcement actions.

3.1.4 Violation Management Layer

The Violation Management Layer is responsible for analyzing the classification results generated by the Detection Layer and determining whether a traffic violation has occurred. This layer serves as the decision-making component of the system by evaluating helmet detection outcomes and initiating appropriate actions when violations are identified.

After receiving the classification results, the system checks whether a detected rider has been classified as “Helmet” or “No Helmet.” If the rider is wearing a helmet, the system continues monitoring subsequent frames without generating any alerts. However, if the rider is classified as “No Helmet,” the system recognizes the event as a traffic safety violation and activates the violation handling process.

Upon detecting a violation, the system generates an alert message and records relevant information, including the detection timestamp, classification result, and associated image frame. These records can later be used by traffic authorities for monitoring, reporting, and enforcement purposes. The system is also designed to prevent duplicate alerts for the same violation within a short time period, thereby reducing unnecessary notifications during continuous video processing.

The Violation Management Layer operates automatically without requiring human intervention, ensuring timely identification of helmet violations and supporting efficient traffic law enforcement. The processed violation information is subsequently forwarded to the Storage and Logging Layer for permanent record keeping and future analysis.

3.1.5 Storage and Logging Layer

The Storage and Logging Layer is responsible for maintaining records of all detection activities performed by the system. This layer ensures that helmet detection results, violation information, and processing logs are securely stored for future reference, reporting, and performance evaluation.

Whenever the system completes a detection cycle, the resulting information is recorded in the storage system. The stored data may include the detection timestamp, image or frame identifier, motorcycle detection status, helmet classification result, confidence score, and violation status.

For helmet violation cases, additional information such as captured evidence images and alert records may also be preserved.

The logging mechanism enables administrators and traffic authorities to review historical detection records, analyze violation trends, and generate statistical reports. These records can assist in evaluating traffic safety compliance and measuring the effectiveness of enforcement activities.

The Storage and Logging Layer also supports system maintenance by recording operational events, error messages, and processing activities. Such logs help developers identify system issues, monitor performance, and improve reliability. By maintaining organized and accessible records, this layer plays a vital role in ensuring accountability, traceability, and long-term usability of the Motorcycle Helmet Detection System.

3.2 List of Proposed Modules & Functions

The functionality of the AI-Based Image-Based Motorcycle Helmet Detection System is implemented through a set of specialized modules. Each module is responsible for performing a specific task within the detection pipeline and collaborates with other modules to ensure accurate and efficient system operation.

3.2.1 Image Acquisition Module

This module is responsible for capturing images and video streams from various input sources, including CCTV cameras, webcams, recorded videos, and image files. It serves as the entry point of the system and ensures that visual data is continuously available for processing.

Functions:

capture_frame(source):

Retrieves image frames from a live camera feed or video source and forwards them for preprocessing.

load_image(image_path):

Loads static images from local storage for helmet detection analysis.

validate_input(source):

Verifies the availability and compatibility of the selected input source before processing begins.

3.2.2 Motorcycle Detection Module

This module is responsible for identifying motorcycles and riders within the input image or video frame using the trained YOLO object detection model. It forms the first stage of the detection process.

Functions:**detect_motorcycle(frame):**

Analyzes the input frame and detects motorcycles using the trained YOLO model.

draw_bounding_boxes(frame, detections):

Draws bounding boxes around detected motorcycles and riders for visual representation.

filter_detections(confidence_threshold):

Removes low-confidence detections to improve detection reliability and accuracy.

3.2.3 Helmet Classification Module

The Helmet Classification Module determines whether a detected rider is wearing a helmet. It processes the rider region identified during motorcycle detection and assigns the appropriate classification label.

Functions:**extract_rider_region(frame, detection):**

Extracts the rider area from the detected motorcycle region for further analysis.

classify_helmet(rider_region):

Classifies the rider as “Helmet” or “No Helmet” using the trained deep learning model.

calculate_confidence_score(prediction):

Computes the confidence level associated with the classification result.

3.2.4 Violation Alert Module

This module evaluates helmet classification results and generates alerts whenever a helmet violation is detected. It automates the identification and reporting of traffic safety violations.

Functions:**check_violation(classification_result):**

Determines whether the classification result represents a helmet violation.

generate_alert(violation_data):

Creates an alert notification when a rider is detected without a helmet.

prepare_violation_record(frame, timestamp):

Collects violation-related information and prepares it for storage and reporting.

3.2.5 Result Logging Module

The Result Logging Module is responsible for storing detection results, violation records, and operational logs. It ensures that historical data is preserved for monitoring and future analysis.

Functions:**save_detection_result(result_data):**

Stores detection outcomes and classification results in the database or log files.

store_violation_record(violation_data):

Records helmet violation details, timestamps, and supporting evidence.

generate_report(log_data):

Produces summarized reports of detection activities and helmet violation statistics.

retrieve_logs(filters):

Retrieves stored detection and violation records based on specified search criteria.

3.3 External Libraries & Frameworks

The AI-Based Image-Based Motorcycle Helmet Detection System utilizes several open-source libraries, frameworks, and development tools to support image processing, object detection, model training, and system management. These technologies provide the foundation for implementing accurate and efficient helmet detection functionality.

1. Python

Python serves as the primary programming language for the development of the system. It provides a simple syntax, extensive library support, and strong integration capabilities with artificial intelligence and computer vision frameworks. Python is used for implementing image processing operations, model integration, data handling, and system logic.

2. YOLO (You Only Look Once)

YOLO is the core object detection framework used by the system. It is responsible for detecting motorcycles and identifying helmet and no-helmet classes in images and video streams. YOLO provides high detection accuracy and fast processing speed, making it suitable for real-time traffic monitoring applications.

3. OpenCV

OpenCV is an open-source computer vision library used for image and video processing. It enables the system to capture video streams, extract frames, preprocess images, draw bounding boxes, and display detection results. OpenCV also supports real-time interaction with CCTV cameras and webcams.

4. TensorFlow / PyTorch

TensorFlow and PyTorch are deep learning frameworks used for training, testing, and optimizing object detection models. These frameworks provide the necessary tools for building neural networks, managing datasets, and evaluating model performance during development.

5. Dataset Annotation Tools

Tools such as Roboflow Annotator and Labelling are used to create labeled datasets for model training. These tools allow developers to annotate motorcycles, riders, helmets, and non-helmet instances, enabling the YOLO model to learn accurate object representations.

6. Git and GitHub

Git is used as a version control system to track code modifications and maintain project history. GitHub serves as a collaborative platform for source code management, project backup, and development coordination. These tools help maintain code quality and simplify future updates.

7. Visual Studio Code (VS Code)

Visual Studio Code is used as the primary development environment for writing, testing, and debugging source code. It provides integrated support for Python development, Git integration, extensions, and debugging tools that improve development productivity.

8. CUDA (Optional GPU Support)

CUDA may be utilized during model training to accelerate deep learning computations using NVIDIA GPUs. GPU acceleration significantly reduces training time and improves the efficiency of model development, especially when working with large datasets.

3.4 Coding Conventions

To ensure code readability, maintainability, and consistency throughout the development process, the AI-Based Image-Based Motorcycle Helmet Detection System follows standard software development and Python programming conventions. These guidelines help developers understand, modify, and extend the system efficiently.

Python Coding Standards

The system follows the Python Enhancement Proposal 8 (PEP 8) guidelines for writing clean and structured code. Function names are written using **snake_case**, while class names follow the **PascalCase** naming convention. Meaningful variable names are used to improve code readability and reduce ambiguity.

Examples:

- `detect_motorcycle()`
- `classify_helmet()`
- `HelmetDetectionModel`

Code Organization

The source code is organized into separate modules based on functionality, including image acquisition, preprocessing, detection, classification, violation management, and logging. This modular structure improves maintainability and allows independent development and testing of individual components.

Comments and Documentation

Developers are encouraged to include comments for complex logic and processing steps. Functions and modules should contain descriptive documentation explaining their purpose, input parameters, and expected outputs. Proper documentation facilitates future maintenance and system enhancements.

Indentation and Formatting

A standard indentation of four spaces is used throughout the project to maintain consistency and comply with Python coding standards. Unnecessary code duplication is avoided, and formatting is kept uniform across all modules.

Error Handling

Appropriate exception handling mechanisms are implemented to manage runtime errors, invalid inputs, camera failures, and file access issues. Error messages are designed to assist developers in identifying and resolving problems quickly.

4. Data Model Schema

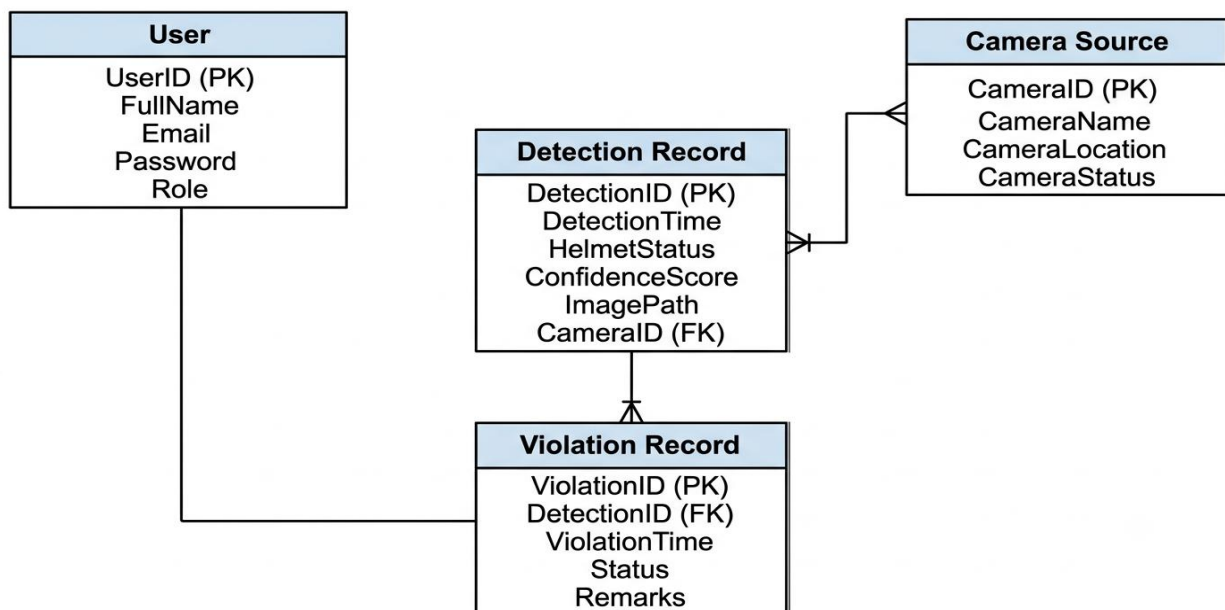
The Data Model Schema defines how detection data, violation records, and system information are organized and stored within the Motorcycle Helmet Detection System. Although the primary functionality of the system is based on computer vision and deep learning, a structured data model is required to maintain detection history, store violation records, and support reporting activities.

The database is designed to ensure efficient storage, retrieval, and management of information generated during helmet detection operations. The schema consists of several entities that represent users, detection records, violation records, and camera sources. These entities are connected through logical relationships that enable effective tracking and monitoring of helmet violations.

4.1 Database Overview

The database serves as the central repository for storing information generated by the helmet detection system. It maintains records of detected motorcycles, helmet classification results, violation events, timestamps, and camera sources. The stored information can be used for administrative monitoring, report generation, and future analysis of traffic safety trends.

The database structure is designed to minimize redundancy while ensuring data integrity and consistency. Each entity stores a specific category of information and interacts with other entities



through defined relationships.

Figure 4.1: Database Architecture Diagram

4.2 Entity Descriptions

4.2.1 User Entity

The User entity stores information about system administrators and authorized personnel who access the system. It is used for authentication, authorization, and system management purposes.

Attributes:

Attribute	Description
UserID (PK)	Unique identifier for each user
FullName	Name of the user
Email	User email address
Password	Encrypted password
Role	User role within the system

Purpose:

This entity allows authorized users to manage system operations, review detection records, and monitor helmet violation activities.

4.2.2 Detection Record Entity

The Detection Record entity stores information generated whenever the system processes an image or video frame. It serves as the primary source of detection history.

Attributes:

Attribute	Description
DetectionID (PK)	Unique detection identifier
DetectionTime	Date and time of detection
HelmetStatus	Helmet or No Helmet classification
ConfidenceScore	Model confidence value
ImagePath	Location of stored image evidence
CameraID (FK)	Associated camera source

Purpose:

This data enables administrators to review previous detections and evaluate system performance.

4.2.3 Violation Record Entity

The Violation Record entity maintains information regarding helmet violations identified by the system.

Attributes:

Attribute	Description
ViolationID (PK)	Unique violation identifier
DetectionID (FK)	Associated detection record
ViolationTime	Time of violation occurrence
Status	Open or Resolved
Remarks	Additional notes regarding the violation

Purpose:

This entity allows tracking and management of helmet violation incidents for reporting and enforcement purposes.

4.2.4 Camera Source Entity

The Camera Source entity stores information about cameras connected to the monitoring system.

Attributes:

Attribute	Description
CameraID (PK)	Unique camera identifier
CameraName	Name of the camera
CameraLocation	Installation location
CameraStatus	Active or Inactive

Purpose:

This entity helps identify the source of detection events and supports monitoring across multiple surveillance locations.

4.3 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) illustrates how the database entities interact with one another. The User entity manages system activities, Detection Records are generated from camera sources, and Violation Records are created whenever a helmet violation is detected. These relationships ensure proper organization of detection and violation data while maintaining data consistency throughout the system.

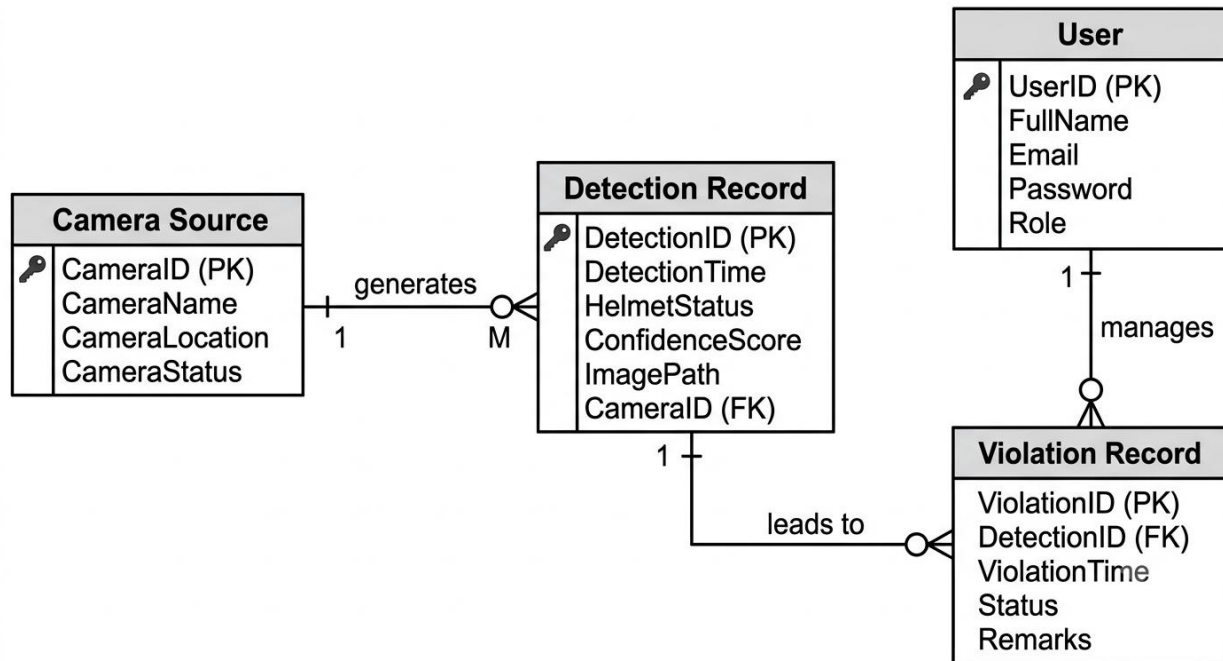


Figure 4.2: Entity Relationship Diagram (ERD)

5. User Interface Design

Design Philosophy

The user interface of the AI-Based Image-Based Motorcycle Helmet Detection System is designed to be simple, responsive, and easy to operate. The primary objective of the interface is to enable administrators and traffic monitoring personnel to efficiently observe live detection results, review violation records, and generate reports without requiring extensive technical knowledge.

Layout

The application utilizes a clean and organized layout that presents critical information in a clear and structured manner. Important statistics, alerts, and monitoring tools are positioned prominently to allow quick access and efficient decision-making.

Navigation

A simple navigation menu is provided to enable users to switch between system modules, including the Dashboard, Live Monitoring, Detection Records, Violation Logs, Reports, and Settings pages. The navigation structure is designed to minimize user effort and improve usability.

Responsiveness

The interface is designed to adapt to different screen sizes, including desktop computers, laptops, tablets, and mobile devices. This ensures that system administrators can monitor helmet detection activities from various platforms without compromising usability.

Visual Design

A professional color scheme and modern user interface components are used to enhance readability and user experience. Detection alerts and violation notifications are visually highlighted to draw immediate attention when safety violations occur.

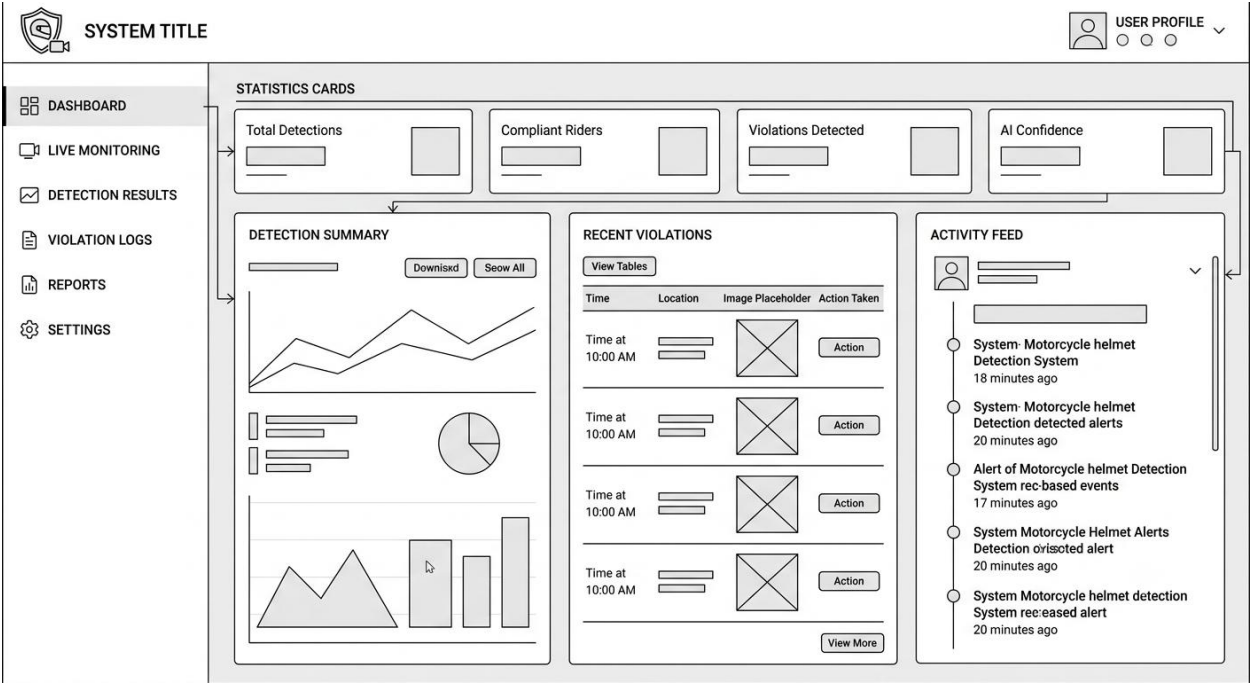


Figure 5.1: Overall User Interface Layout

5.1 List of Core Pages & Control Details

1. Dashboard Screen

Description:

The Dashboard serves as the main page of the system and provides an overview of helmet detection activities and system statistics.

Controls:

- Summary cards displaying Total Detections.
- Summary cards displaying Total Violations.
- Detection accuracy statistics.
- Recent activity panel.
- Quick access buttons for monitoring and reports.

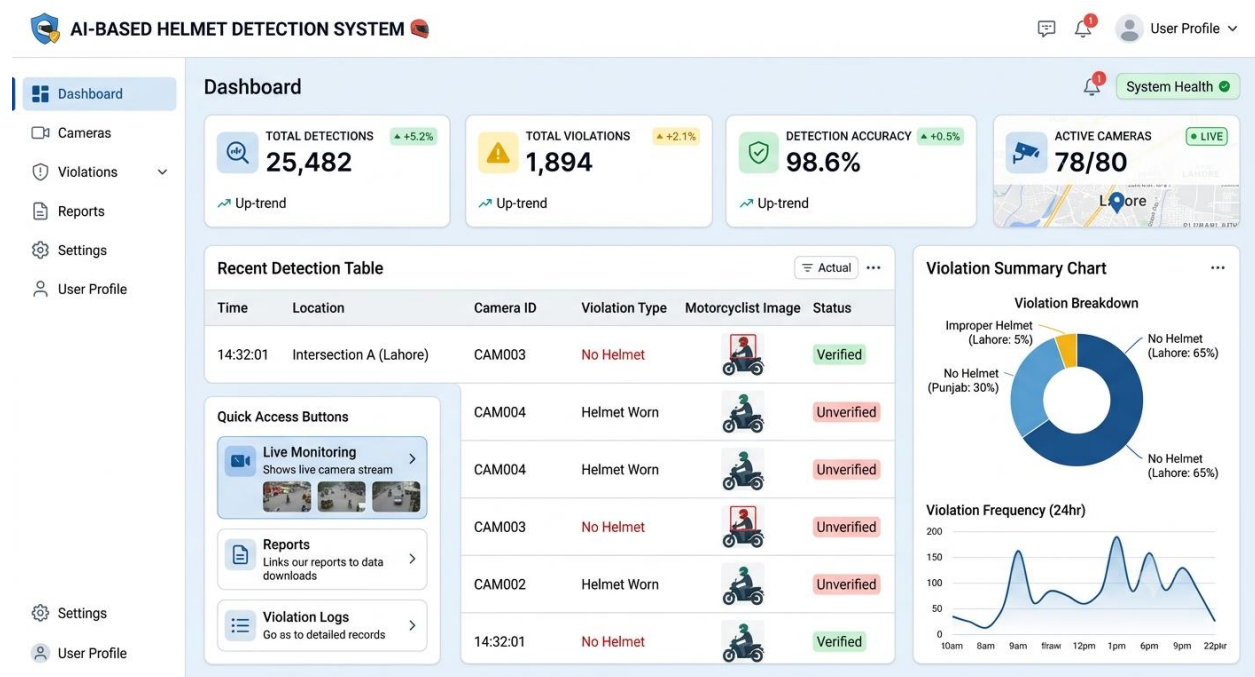


Figure 5.2: Dashboard Screen Mockup

2. Live Monitoring Screen

Description:

This screen displays real-time video streams from connected cameras and shows helmet detection results as they occur.

Controls:

- Live camera feed display.
- Start Monitoring button.
- Stop Monitoring button.
- Detection status indicators.

Real-time violation notifications.

3. Detection Results Screen

Description:

This page displays all processed detections along with classification results and confidence scores.

Controls:

- Detection records table.
- Search functionality.
- Date-based filtering options.
- Detection image preview.
- Confidence score display.

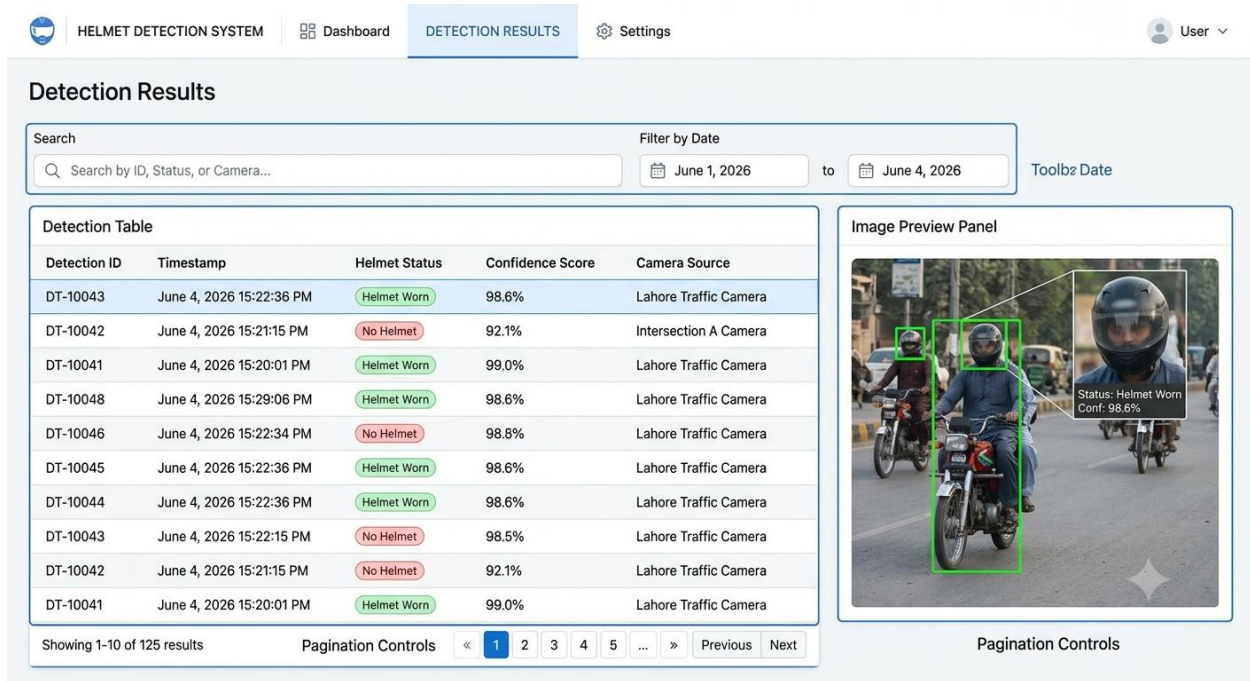


Figure 5.4: Detection Results Screen Mockup

4. Violation Logs Screen

Description:

The Violation Logs page provides detailed information regarding helmet violation incidents detected by the system.

Controls:

- Violation records table.
- Timestamp display.
- Violation status indicators.
- Search and filter options.
- Detailed violation information panel.

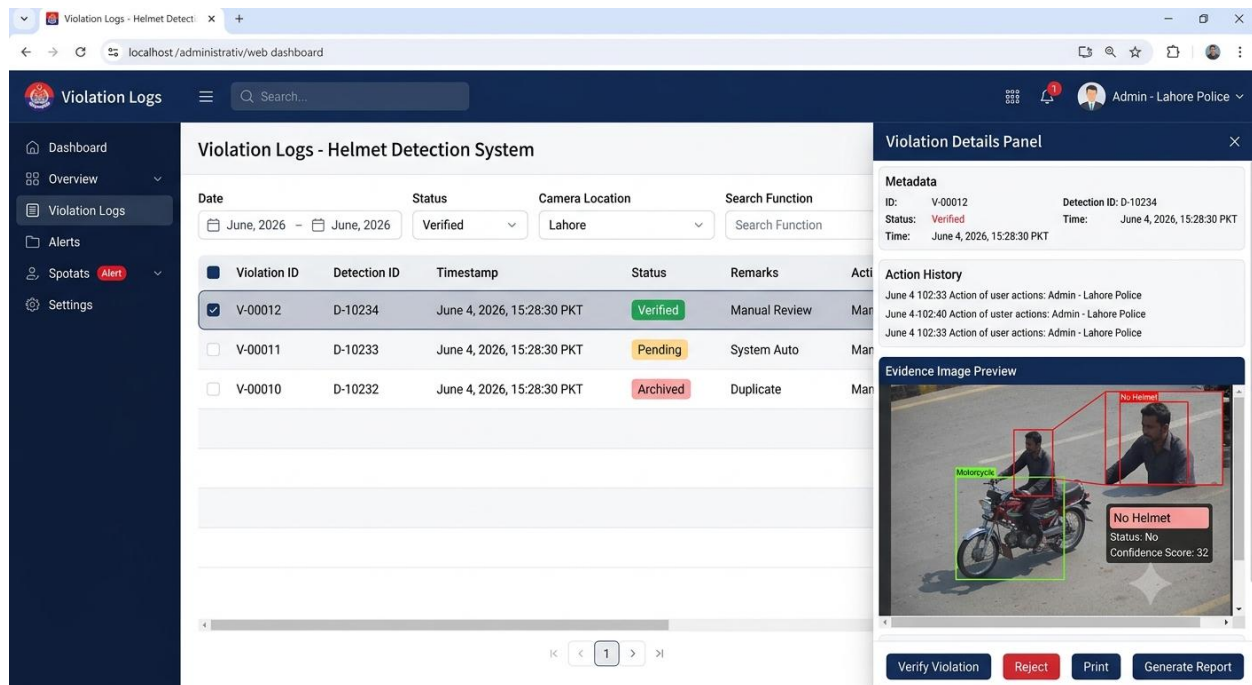


Figure 5.5: Violation Logs Screen Mockup

5. Reports Screen

Description:

This page allows administrators to review system statistics and generate reports related to helmet detection activities.

Controls:

- Statistical summary tables.
- Date range selection.
- Report generation button.
- Download PDF button.
- Violation trend charts.

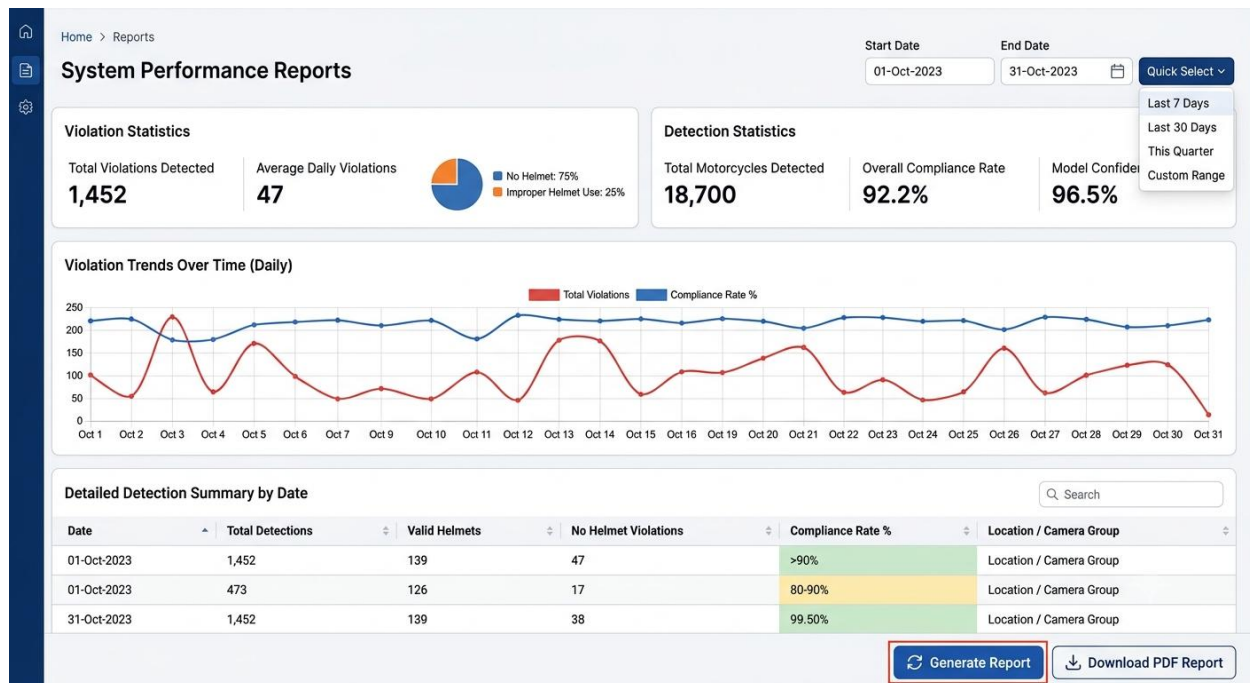


Figure 5.6: Reports Screen Mockup

6. Settings Screen

Description:

The Settings page allows administrators to configure cameras, detection parameters, and system preferences.

Controls:

- Camera configuration settings.
- Detection threshold settings.
- User management controls.
- Save Settings button.
- Reset Settings button.

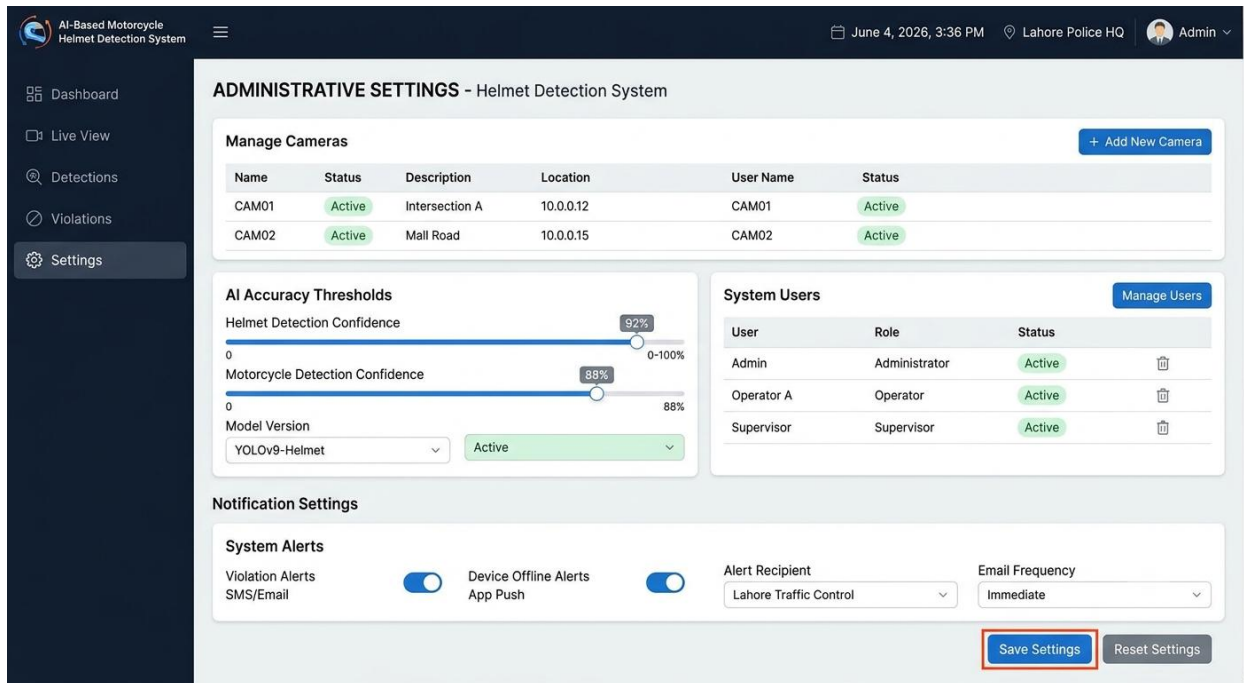


Figure 5.7: Settings Screen Mockup

5.2 Input Constraints & Validation Rules

To maintain data integrity and ensure reliable system operation, the following validation rules are implemented throughout the application.

Empty Fields

The system displays validation messages whenever mandatory input fields are left blank. Users must provide all required information before proceeding.

Data Type Validation

Text fields only accept textual data, while numerical fields only accept valid numeric values. Invalid data types are automatically rejected.

Email Validation

User email addresses must follow a valid email format before account creation or system access is permitted. This ensures that only properly structured and deliverable email addresses are accepted by the system. It also helps prevent invalid registrations and supports secure user authentication and communication.

Camera Validation

The system verifies camera availability before initiating real-time monitoring. Invalid or disconnected camera sources generate appropriate warning messages. This ensures that the monitoring process only starts when a valid and active camera feed is detected, preventing runtime failures and data loss during operation. It also improves system reliability by handling hardware or connection issues at an early stage.

Detection Threshold Validation

Detection confidence thresholds must remain within acceptable predefined ranges to prevent invalid configuration settings. This ensures that the system maintains a balance between accuracy and performance during helmet detection. It also helps avoid overly strict or overly loose detection behavior, which could lead to false positives or false negatives in real-time monitoring.

File Format Validation

Uploaded images and videos must be in supported file formats. Unsupported file types are rejected to prevent processing errors. This ensures that only compatible media is processed by the system, reducing the risk of runtime failures and improving overall processing efficiency. It also helps maintain consistency in input data for accurate detection results.

By implementing these validation rules, the system ensures accurate data entry, reliable operation, and improved user experience.

6. References

- [1] Software Requirements Specification (SRS), *AI-Based Image-Based Motorcycle Helmet Detection System*, Department of Computer Science, The Islamia University of Bahawalpur, 2025.
- [2] Joseph Redmon et al., *YOLO (You Only Look Once): Unified Real-Time Object Detection*, Computer Vision and Pattern Recognition (CVPR), 2016.
- [3] Ultralytics, *YOLO Documentation and Implementation Guide*.
- [4] OpenCV Development Team, *Open Source Computer Vision Library (OpenCV) Documentation*.
- [5] TensorFlow Documentation, Google Developers.
- [6] PyTorch Documentation, Meta AI Research.
- [7] Roboflow, *Dataset Annotation and Computer Vision Tools Documentation*.
- [8] Kaggle, *Computer Vision and Helmet Detection Datasets*.
- [9] Python Software Foundation, *Python Programming Language Documentation*.
- [10] Git and GitHub Documentation, Version Control and Collaborative Software Development Resources.
- [11] IEEE Software Engineering Standards and Documentation Guidelines.
- [12] Pressman, R. S., *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.